

Disciplina:

MACHINE LEARNING

Professor: Murilo A. R. Bigoto



PAUTA

O que buscamos com os modelos de ML?

Algoritmos Supervisionados
Regressão e Classificação

Algoritmos Não Supervisionados

EXEMPLO PRÁTICO

[Relembrando] Criação de Features

Matemática: Razões, diferenças e transformações logarítmicas/polinomiais

Sazonalidade: Extração de componentes de data/hora (seno/cosseno para ciclicidade)

Agregações: Métricas de comportamento acumulado (ex: média de gastos 30d)

Séries Temporais: Lag, dia da semana, janelas móveis

[Prompt] Criação de Features

Aja como um Especialista em Feature Engineering. Eu tenho um DataFrame e meu objetivo é construir um modelo de Machine Learning para [INSERIR OBJETIVO, EX: prever o churn de clientes].

1. Estrutura dos Dados:

O dataset possui as seguintes colunas: [LISTAR COLUNAS PRINCIPAIS OU COLAR O `df.info()`].

A unidade de observação (cada linha) representa: [EX: um cliente por mês].

2. Desafio:

Preciso que você sugira 10 novas features que possam aumentar o poder preditivo do modelo, divididas nas seguintes categorias:

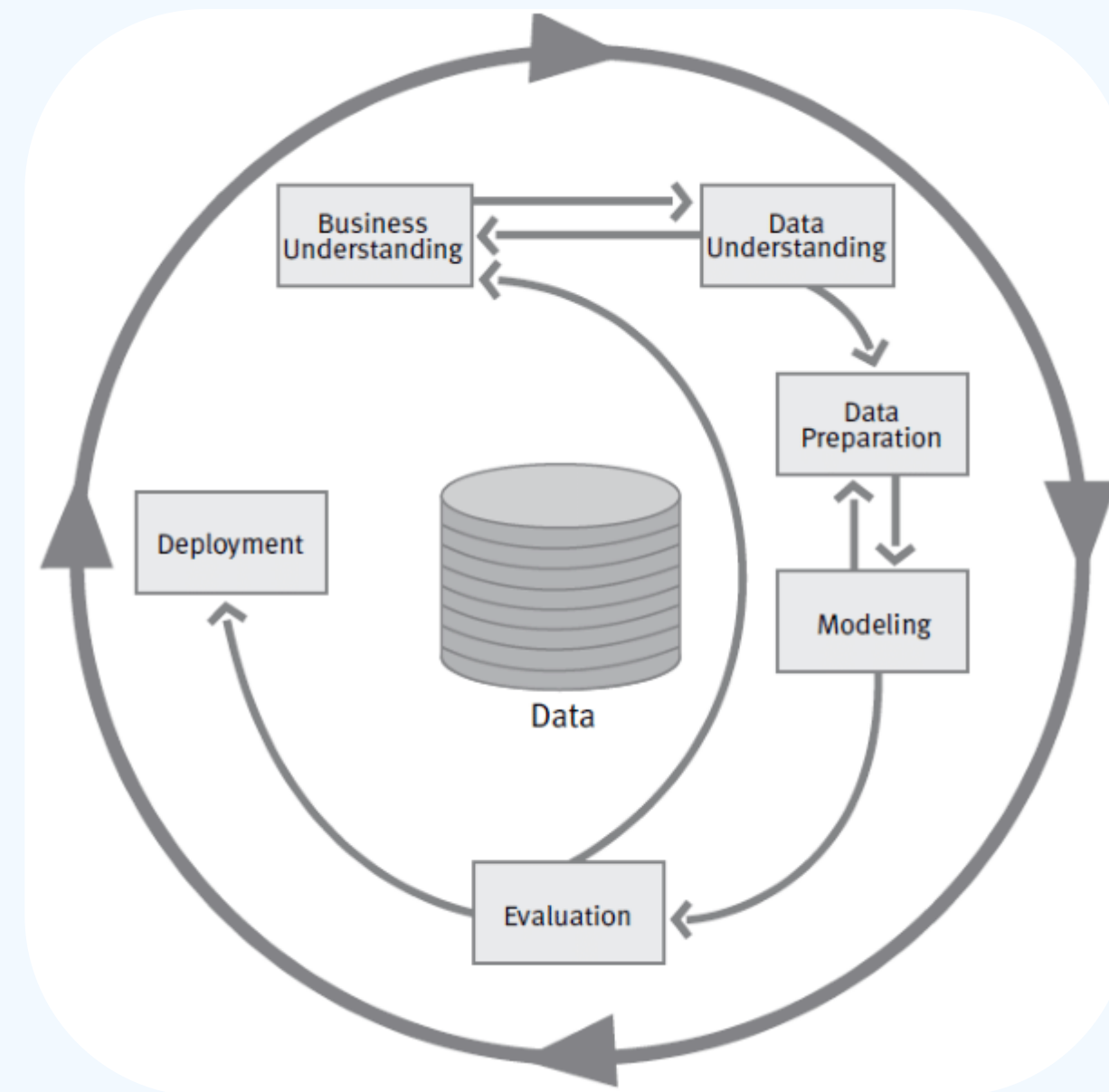
- Transformações Matemáticas: (Logs, razões, polinômios).
- Agregações de Negócio: (Médias móveis, contagens, frequência).
- Features de Tempo: (Sazonalidade, tempo desde o último evento).
- Interações: (Combinação de duas ou mais variáveis existentes).

3. Formato de Saída:

Para cada sugestão, forneça:

- Nome da Feature.
- Raciocínio Lógico: Por que isso ajudaria o modelo a entender o fenômeno?
- Snippet de Código: Em Python/Pandas para criar a coluna.

Ciclo de Vida de um Projeto (CRISP-DM)



Cross-Industry Standard Process for Data Mining

Objetivo dos Algoritmos

Em Machine Learning, assumimos que existe uma função f que mapeia as variáveis de entrada (X) para a saída (y).

Como essa função é desconhecida, o algoritmo busca, dentro de um espaço de possibilidades, uma hipótese que melhor se ajuste aos dados observados.

Dizer que o modelo é uma "hipótese de ajuste" significa que estamos testando uma suposição matemática sobre como os dados se comportam. O objetivo do treinamento é encontrar os parâmetros que minimizam a distância entre a nossa hipótese e o fenômeno real.

$$y = h_{\theta}(x) + \epsilon$$

Objetivo dos Algoritmos

$$y = h_{\theta}(x) + \epsilon$$

- y : É a variável alvo (o valor real que queremos prever).
- x : É o vetor de características (features) de entrada.
- h_{θ} : É a nossa hipótese, uma função parametrizada por um vetor de pesos θ .
- e : Representa o erro aleatório (ruído) que não pode ser explicado pelo modelo.

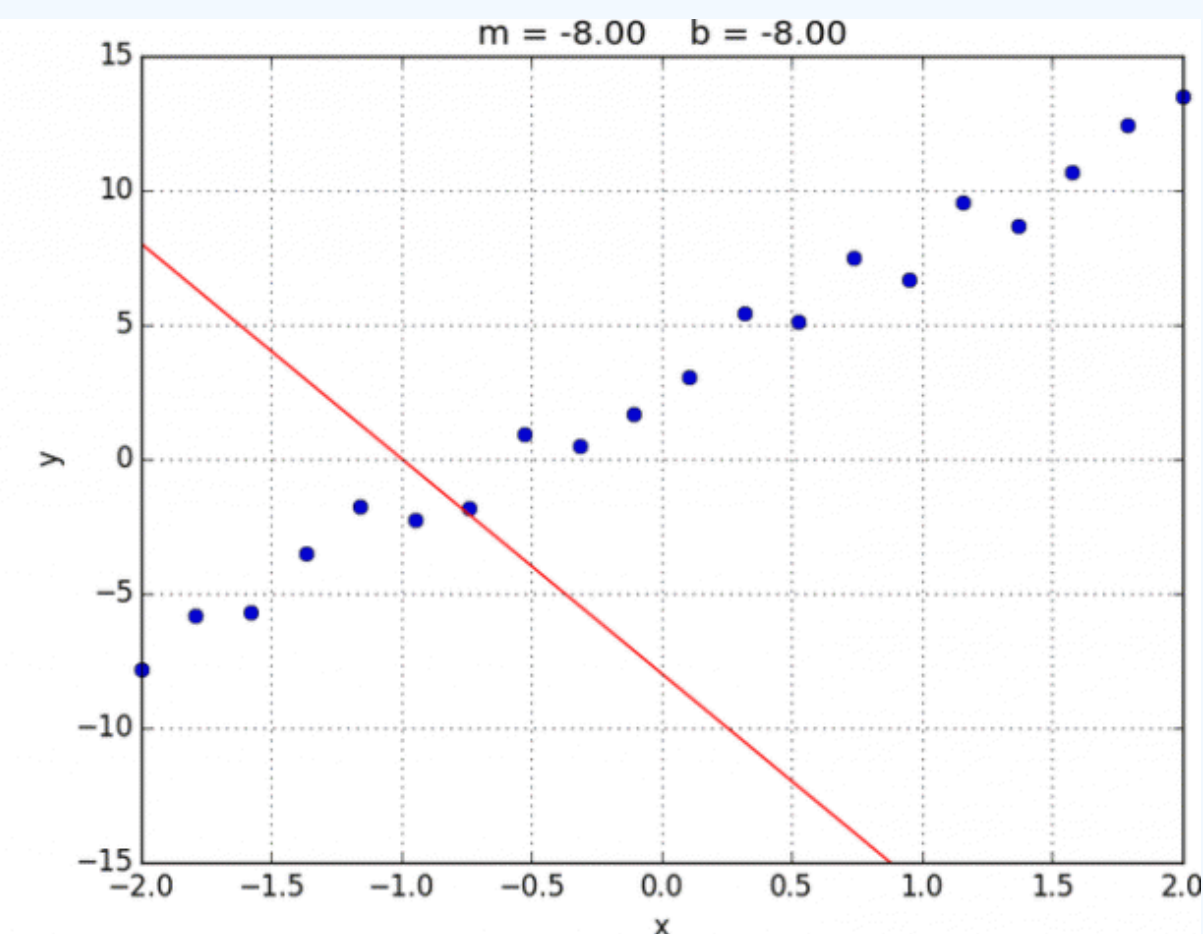
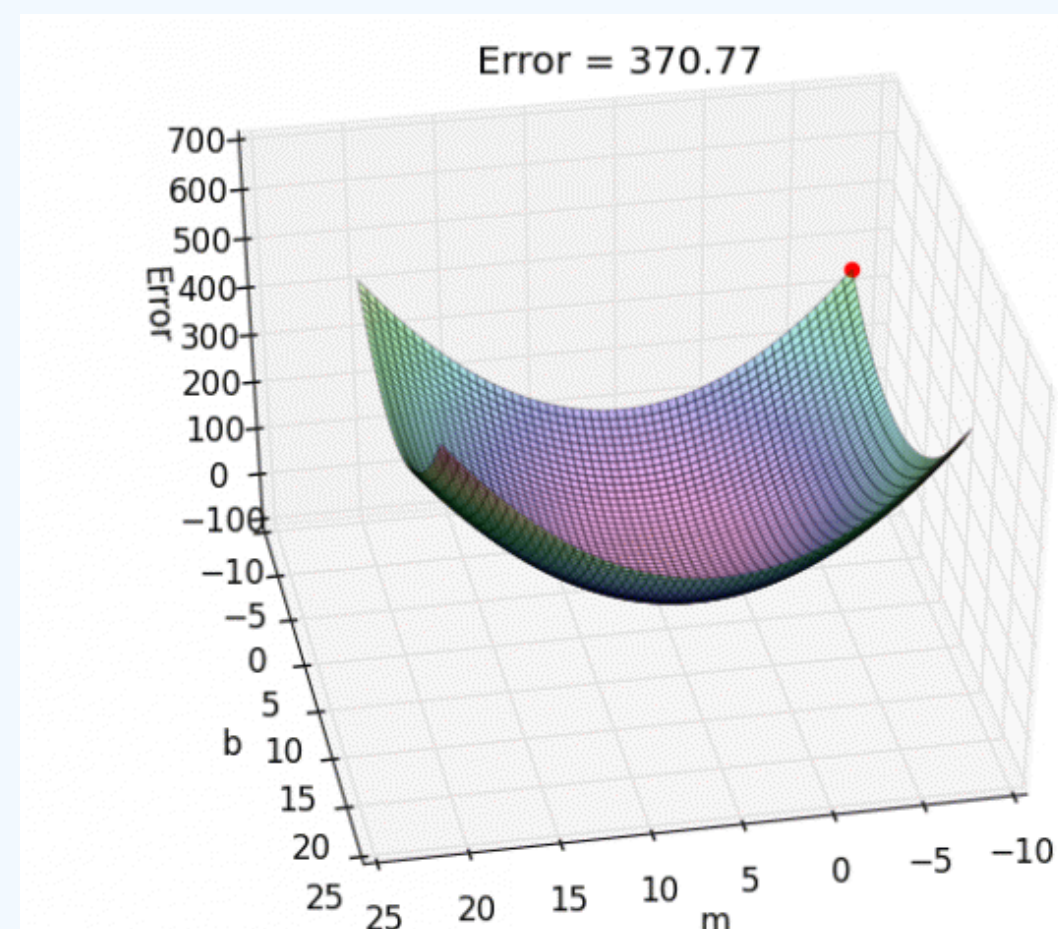
Objetivo dos Algoritmos

Para que essa hipótese seja útil, precisamos resolver um problema de otimização. O "ajuste aos dados" é a busca pelo melhor conjunto de parâmetros θ^* que minimiza uma Função de Perda (J):

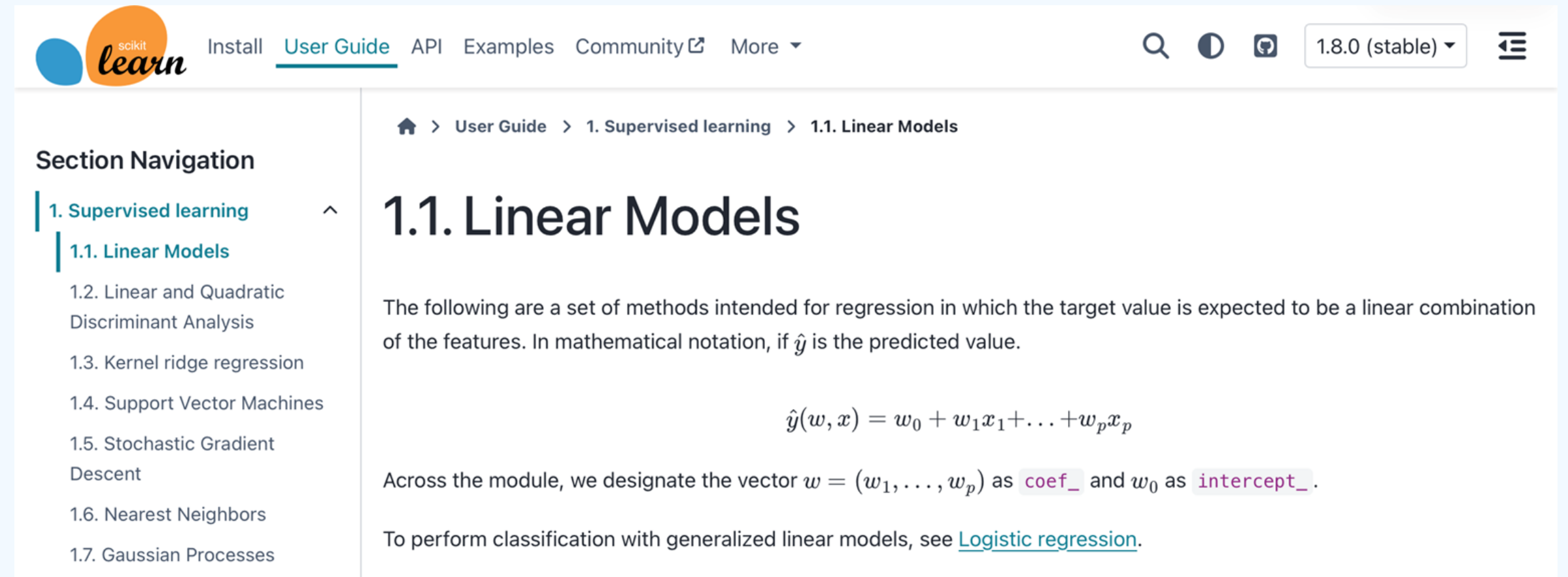
$$\theta^* = \arg \min_{\theta} J(\theta)$$

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Objetivo dos Algoritmos



Regressões



The screenshot shows the scikit-learn User Guide page for Linear Models. The page has a navigation bar at the top with links for Install, User Guide, API, Examples, Community, and More. The User Guide link is highlighted. On the right side of the navigation bar, there is a search icon, a moon icon, a GitHub icon, a version dropdown menu set to 1.8.0 (stable), and a hamburger menu icon. The main content area is titled "1.1. Linear Models" and includes a breadcrumb trail: Home > User Guide > 1. Supervised learning > 1.1. Linear Models. The text explains that the following methods are intended for regression where the target value is a linear combination of features. It provides the mathematical notation for the predicted value $\hat{y}(w, x) = w_0 + w_1x_1 + \dots + w_px_p$. It also mentions that $w = (w_1, \dots, w_p)$ is designated as `coef_` and w_0 as `intercept_`. A link to "Logistic regression" is provided for classification with generalized linear models. A sidebar on the left contains a "Section Navigation" menu with a list of topics: 1. Supervised learning (expanded), 1.1. Linear Models (selected), 1.2. Linear and Quadratic Discriminant Analysis, 1.3. Kernel ridge regression, 1.4. Support Vector Machines, 1.5. Stochastic Gradient Descent, 1.6. Nearest Neighbors, and 1.7. Gaussian Processes.

scikit-learn

Install [User Guide](#) API Examples Community [More](#)

1.8.0 (stable)

Section Navigation

- 1. Supervised learning [^](#)
 - 1.1. Linear Models
 - 1.2. Linear and Quadratic Discriminant Analysis
 - 1.3. Kernel ridge regression
 - 1.4. Support Vector Machines
 - 1.5. Stochastic Gradient Descent
 - 1.6. Nearest Neighbors
 - 1.7. Gaussian Processes

1.1. Linear Models

The following are a set of methods intended for regression in which the target value is expected to be a linear combination of the features. In mathematical notation, if $\hat{y}$ is the predicted value.

$$\hat{y}(w, x) = w_0 + w_1x_1 + \dots + w_px_p$$

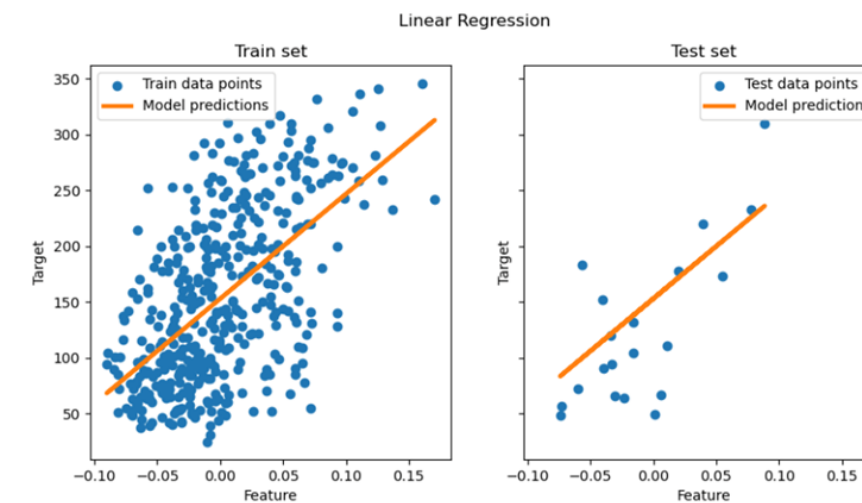
Across the module, we designate the vector $w = (w_1, \dots, w_p)$ as `coef_` and w_0 as `intercept_`.

To perform classification with generalized linear models, see [Logistic regression](#).

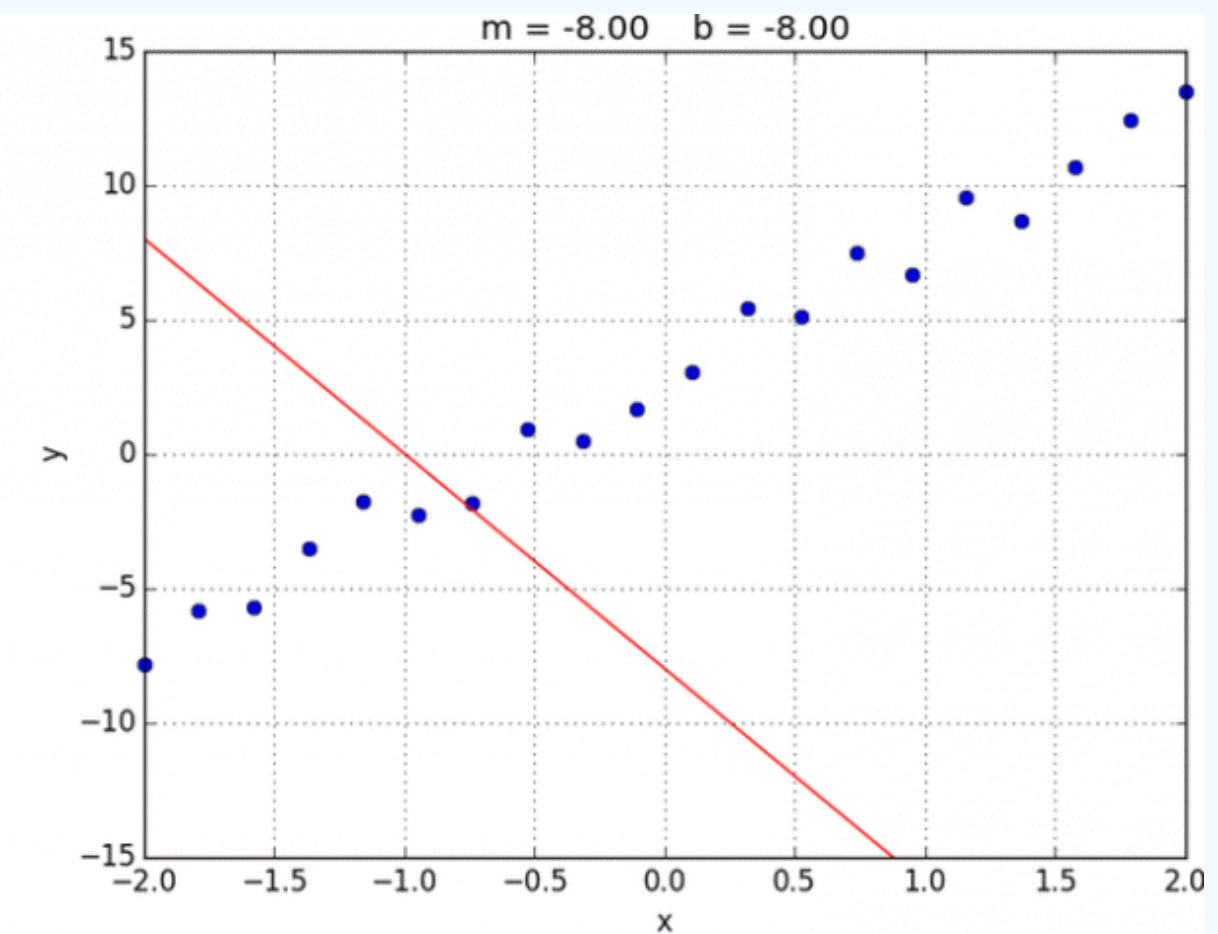
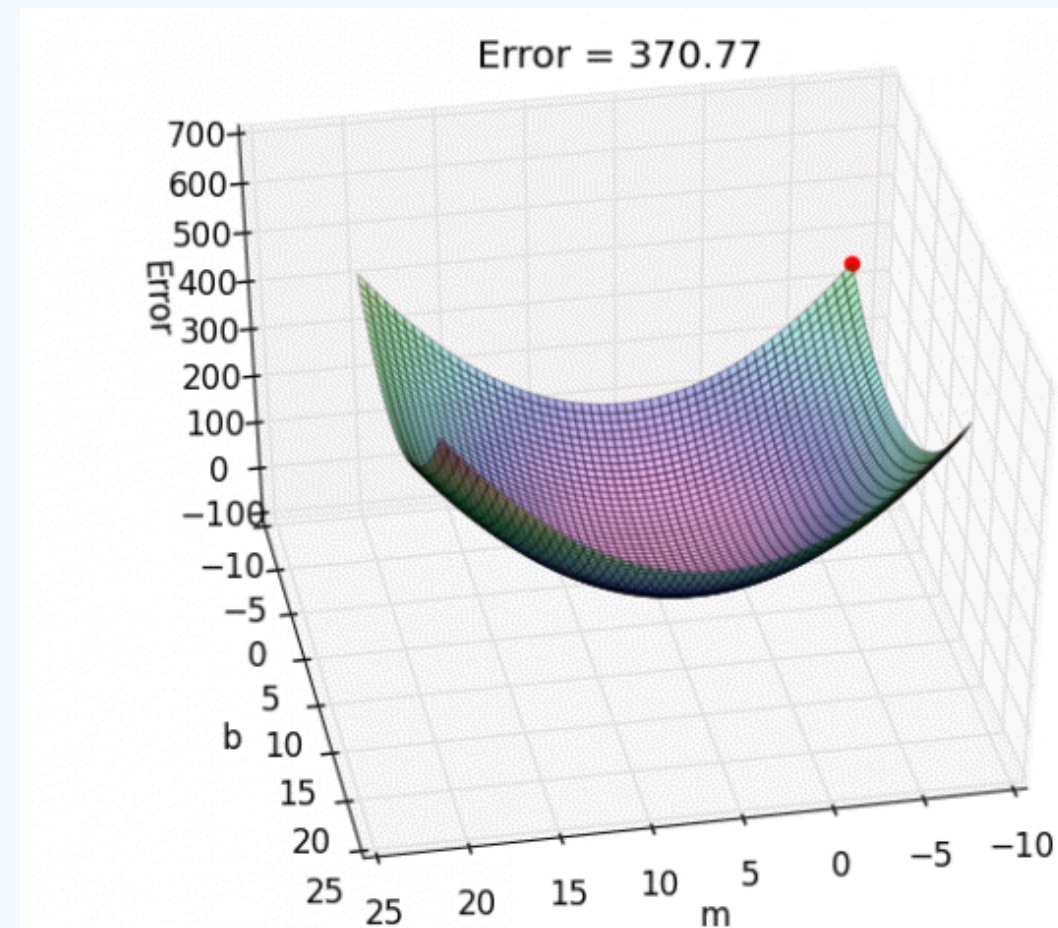
Regressões

LinearRegression fits a linear model with coefficients $w = (w_1, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation. Mathematically it solves a problem of the form:

$$\min_w ||Xw - y||_2^2$$



Regressões



Regressões

python

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 1. Dados de exemplo (X precisa ser 2D)
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 5, 4, 5])

# 2. Instanciar e Treinar o modelo
modelo = LinearRegression().fit(X, y)

# 3. Fazer uma predição
previsao = modelo.predict([[6]])

print(f"Predição para o valor 6: {previsao[0]:.2f}")
print(f"Coeficiente (Angular): {modelo.coef_[0]:.2f}")
print(f"Intercepto (Linear): {modelo.intercept_:.2f}")
```

Regressões Penalidades

Minimiza:

$$\min_w ||y - Xw||^2 + \alpha \cdot \text{penalty}$$

- `alpha` : força da regularização
- Reduz magnitude dos coeficientes (`coef_`)

Regressões Penalidades

◆ Ridge (Ridge) — L2 penalty

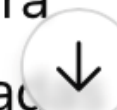
- Penalização: $||w||^2$
- Não zera coeficientes
- Estável com features correlacionadas

◆ Lasso (Lasso) — L1 penalty

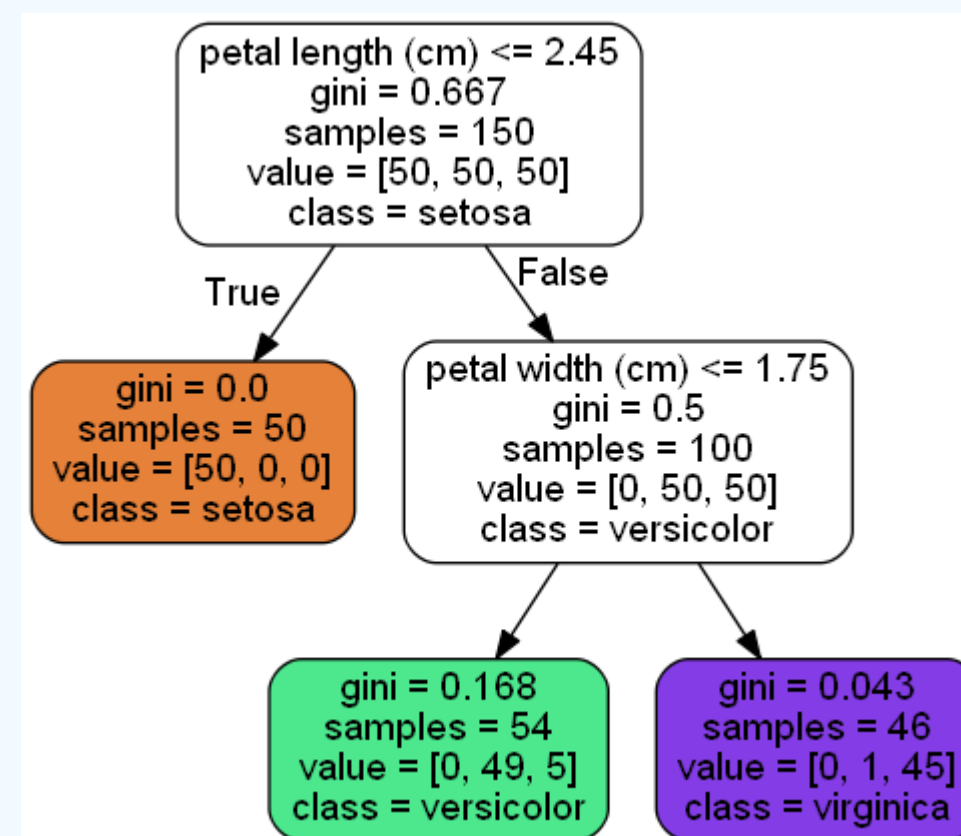
- Penalização: $||w||_1$
- Pode zerar coeficientes
- Faz **feature selection**

◆ ElasticNet (ElasticNet) — L1 + L2

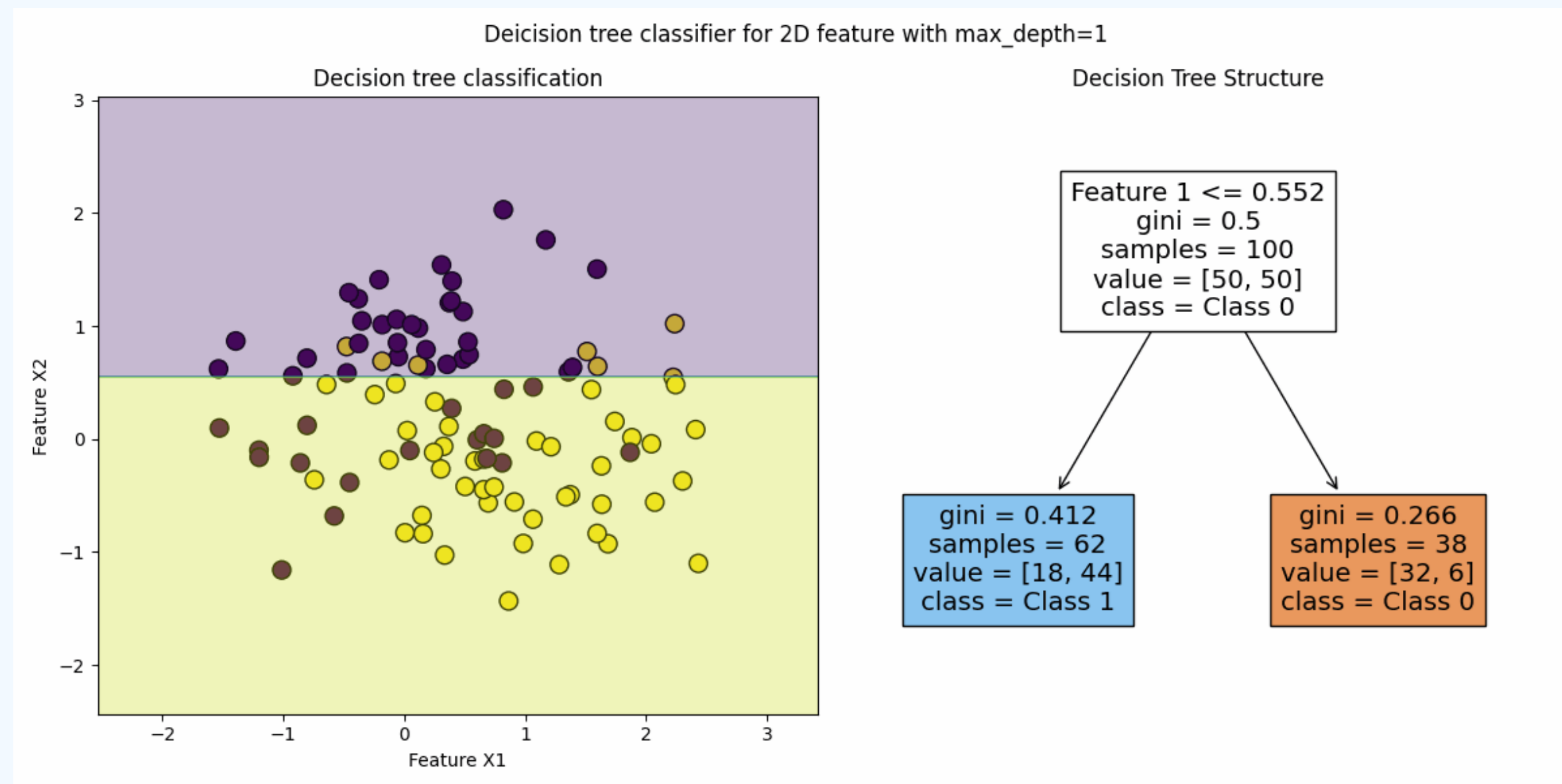
- Combina L1 e L2
- Parâmetro `l1_ratio` controla mistura
- Útil com muitas features correlacionadas



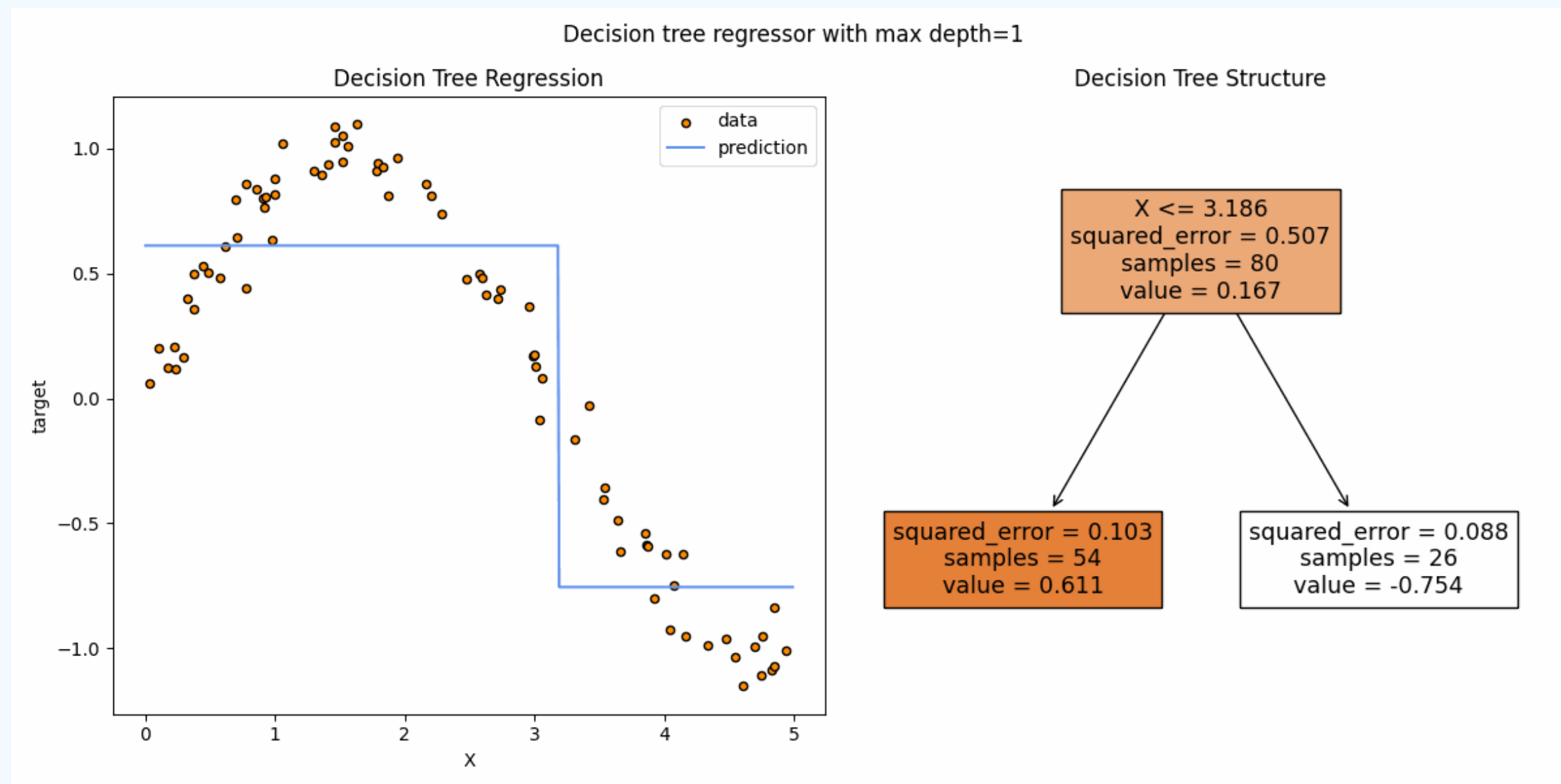
Árvore de Decisão



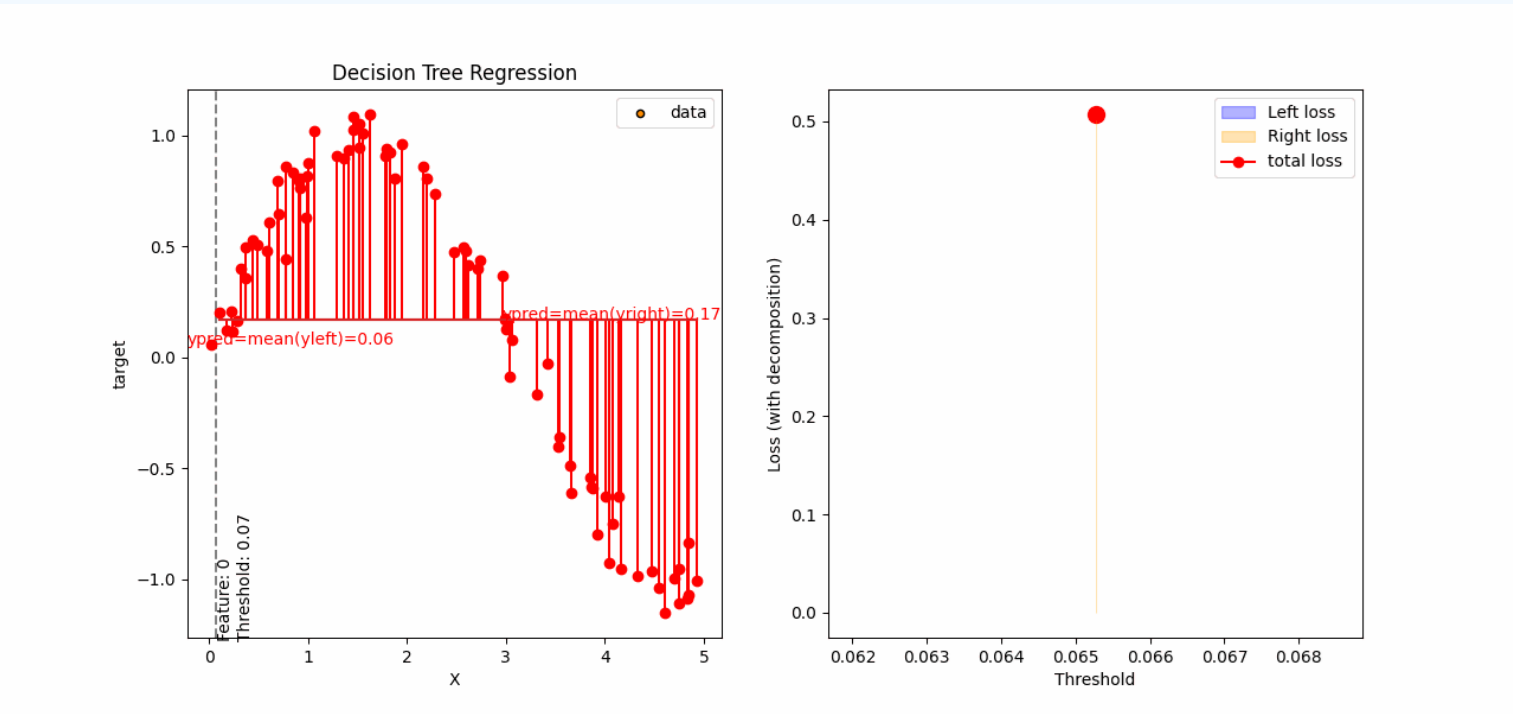
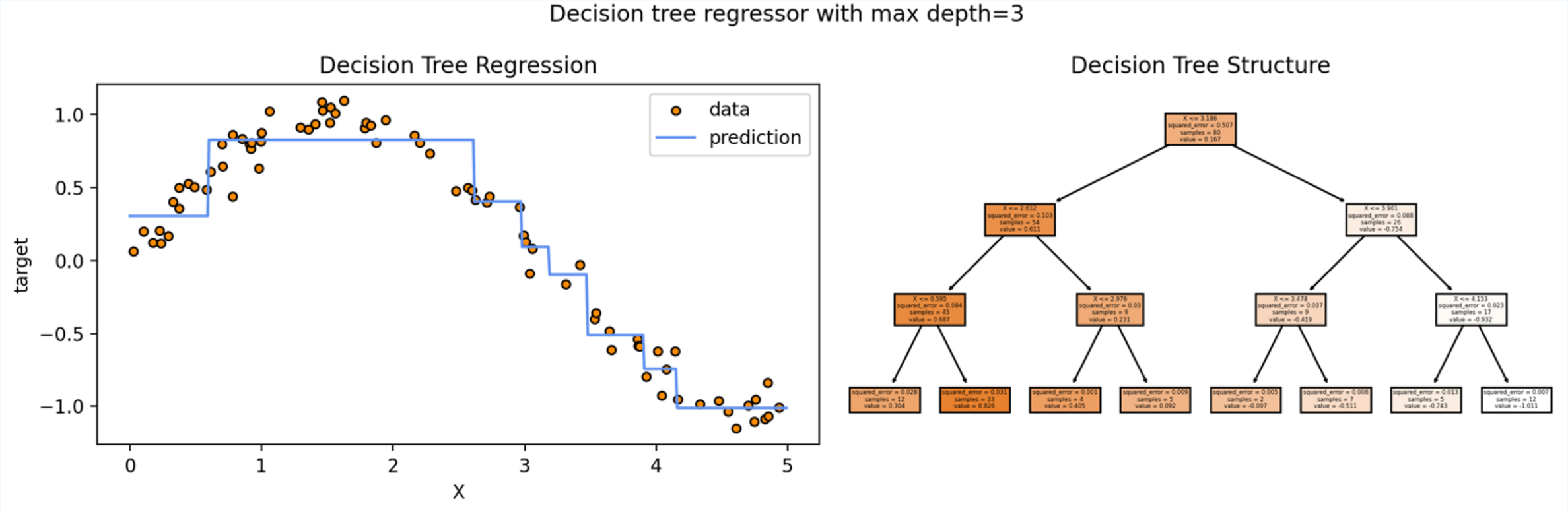
Árvore de Decisão [Classificacao]



Árvore de Decisão [Regressao]

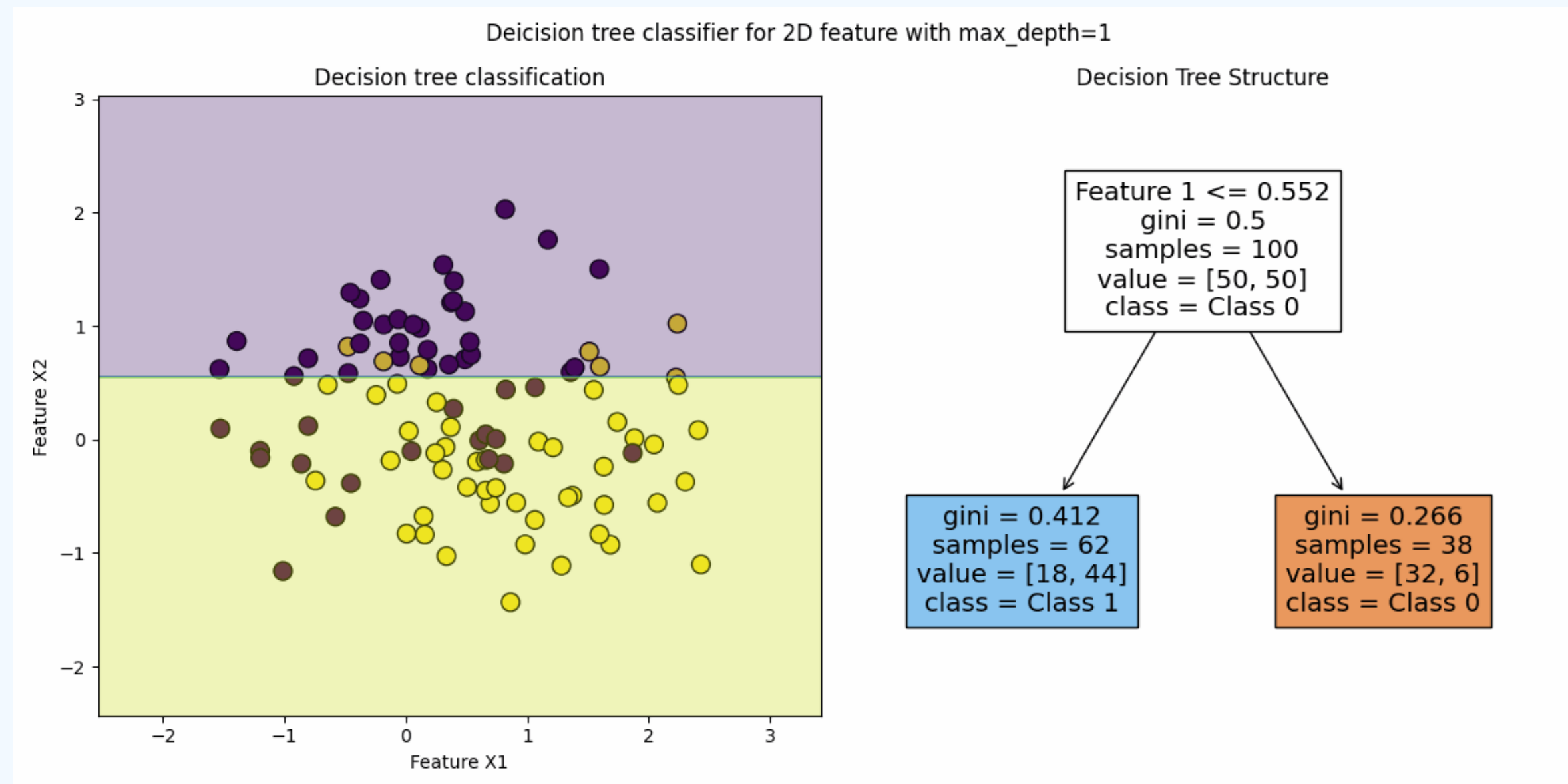


Árvore de Decisão [Regressao]



$$\min_{k,s} \left[\sum_{i:x_{i,k} \leq s} (y_i - \hat{y}_L)^2 + \sum_{i:x_{i,k} > s} (y_i - \hat{y}_R)^2 \right]$$

Árvore de Decisão [Classificacao]



3. A Equação de Otimização do Split

Para escolher a melhor variável X_j e o melhor ponto de corte s , o algoritmo minimiza a soma ponderada das impurezas dos nós resultantes (esquerdo e direito):

$$\min_{j,s} \left[\frac{n_L}{n_m} Q_L + \frac{n_R}{n_m} Q_R \right]$$

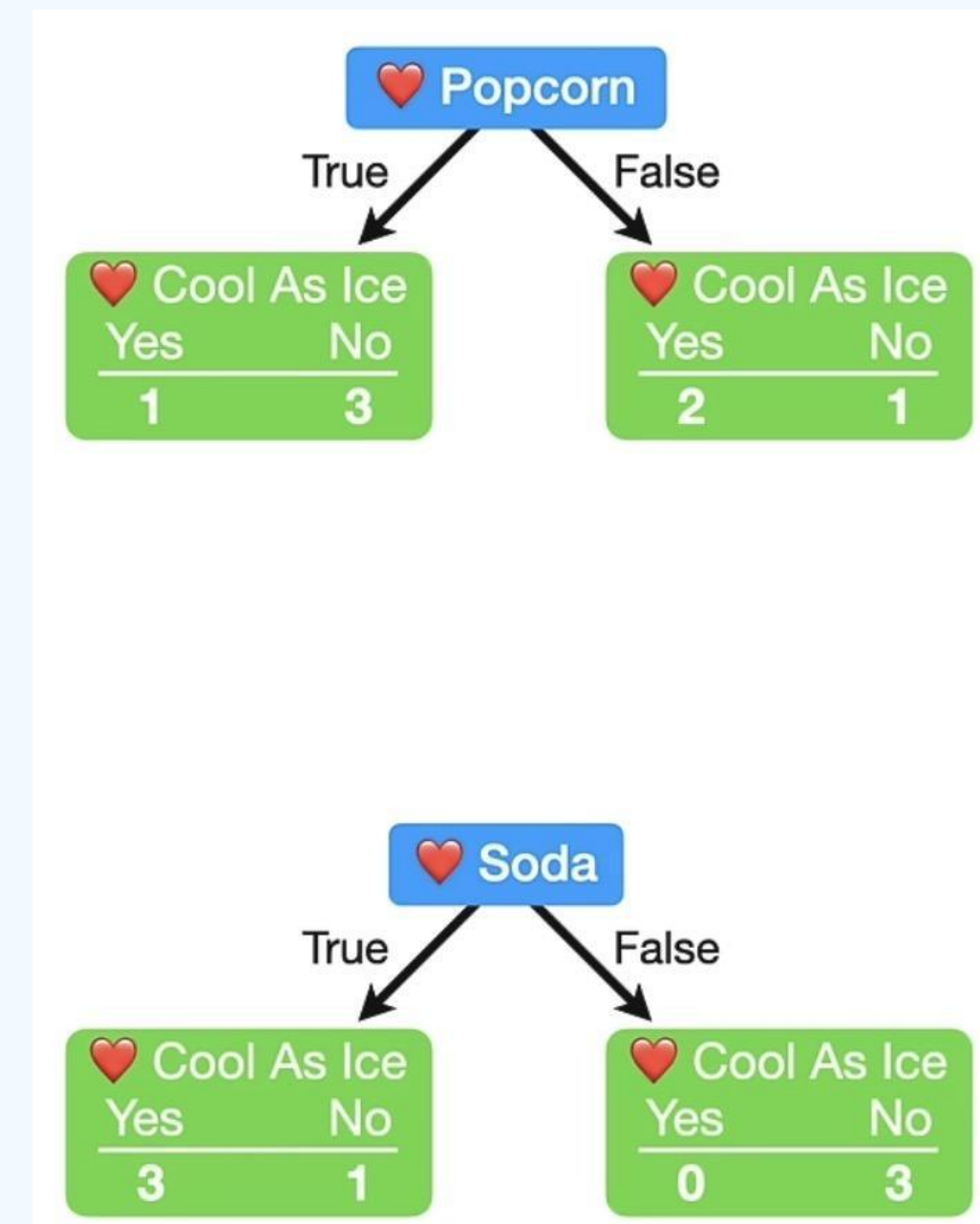
Onde:

- n_L e n_R : Número de observações nos nós filhos (esquerdo e direito).
- n_m : Número total de observações no nó pai.
- Q : A medida de impureza escolhida (Gini ou Entropia).

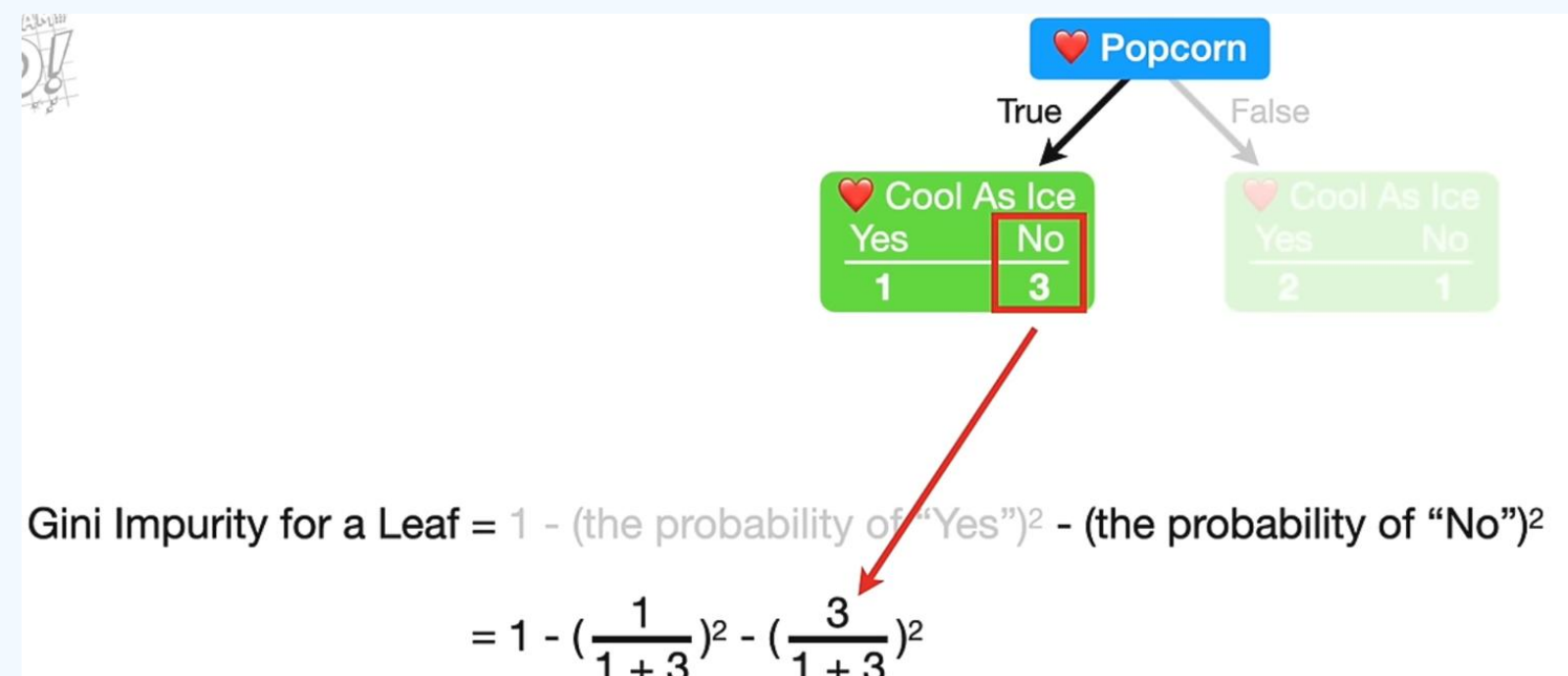
Árvore de Decisão [Classificacao]

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$



Árvore de Decisão [Classificacao]

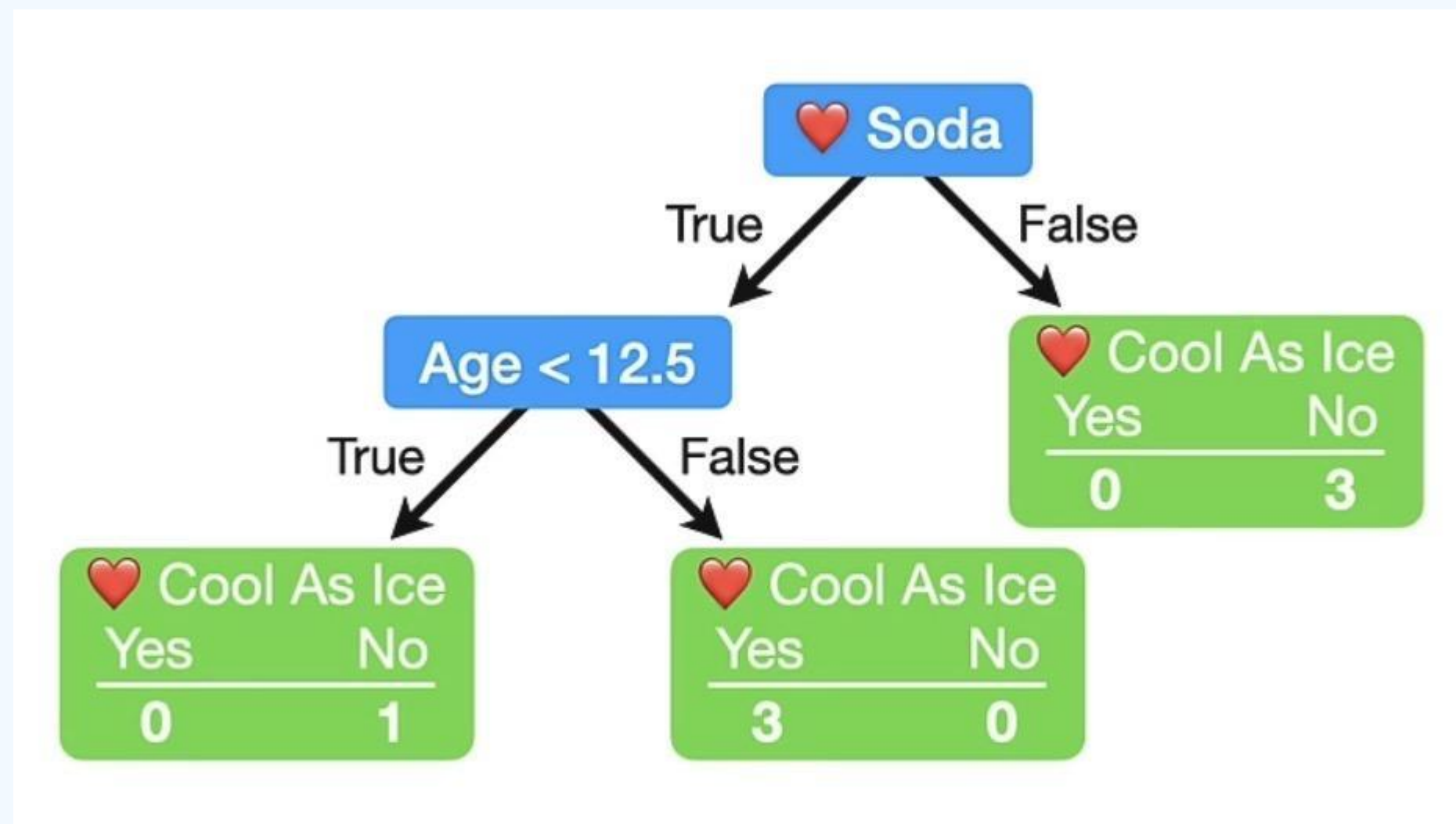


$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

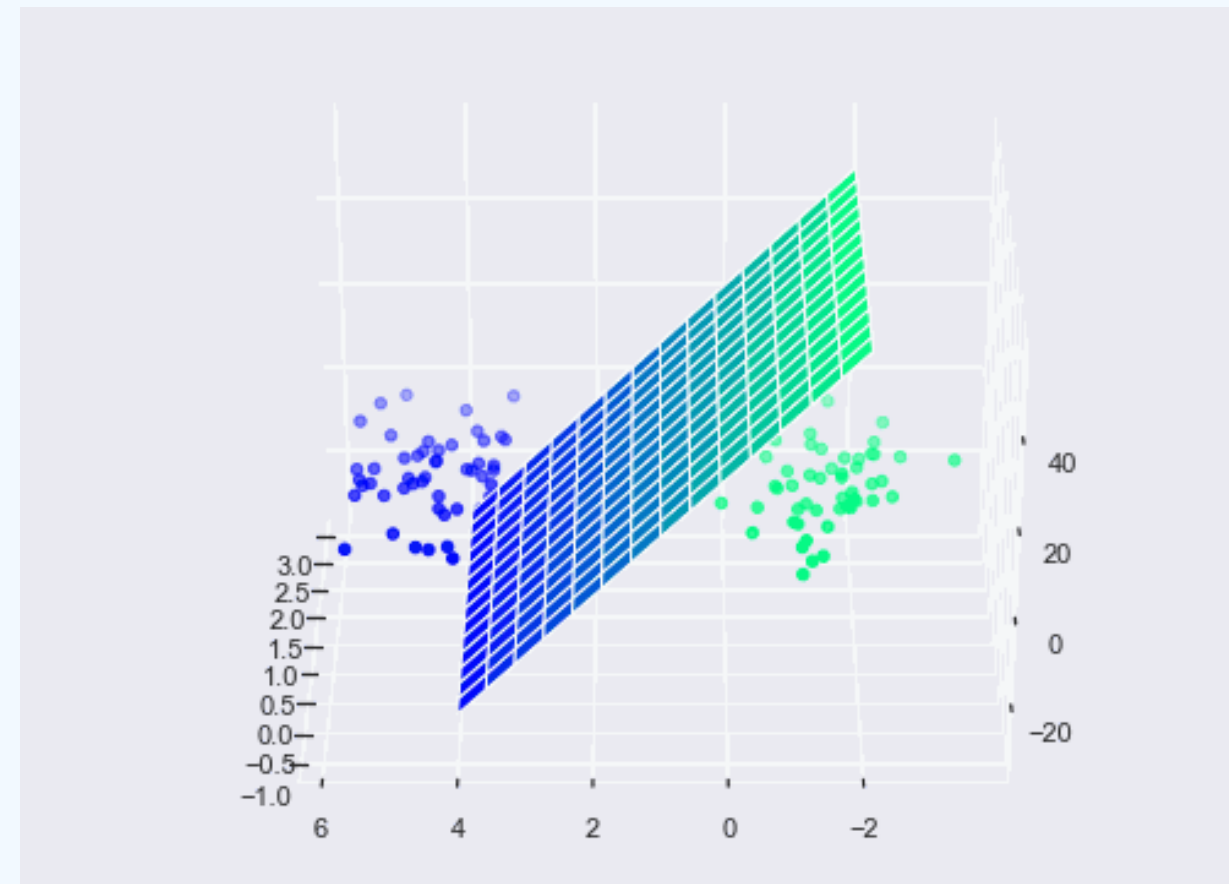
Árvore de Decisão [Classificação]



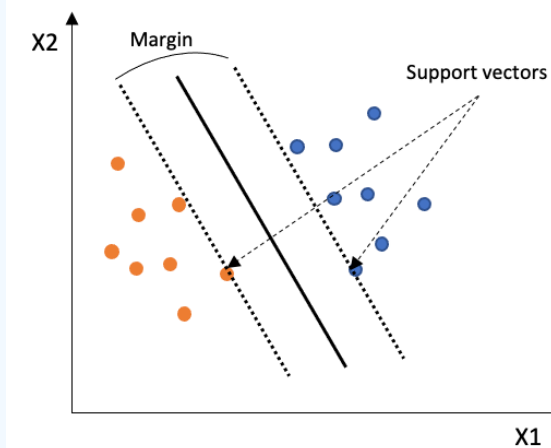
Árvore de Decisão [Classificação]



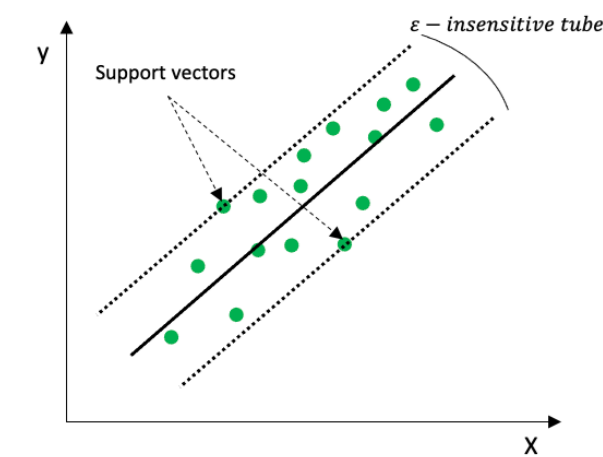
SVM



$$\min_{w,b,\zeta} \frac{1}{2} w^T w + C \sum_{i=1}^n \zeta_i$$

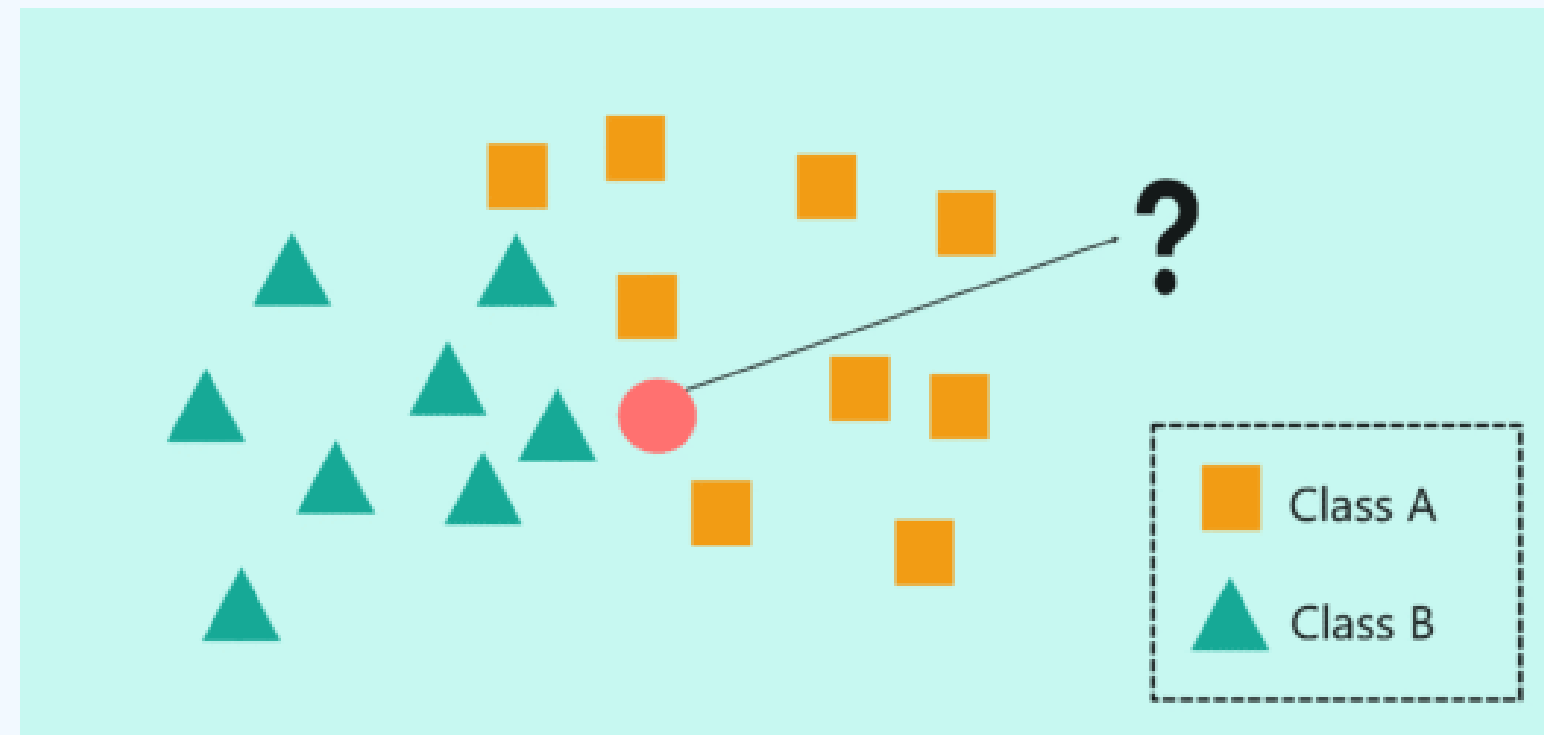


Classification problem using SVM



Regression problem using SVR

KNN



$$d(x, x_i) = \left(\sum_{j=1}^d |x_j - x_{i,j}|^p \right)^{1/p}$$

Regressão Logística

- Busca modelar a probabilidade de uma dada observação pertencer a uma determinada classe;
- Forma de regressão não linear, pois não assume que a relação entre as variáveis independentes e a variável dependente seja linear;
- O objetivo da regressão logística é encontrar os parâmetros da função que melhor explicam os dados de treinamento. Tais parâmetros são encontrados usando um algoritmo de otimização;
- Tem fácil interpretabilidade.

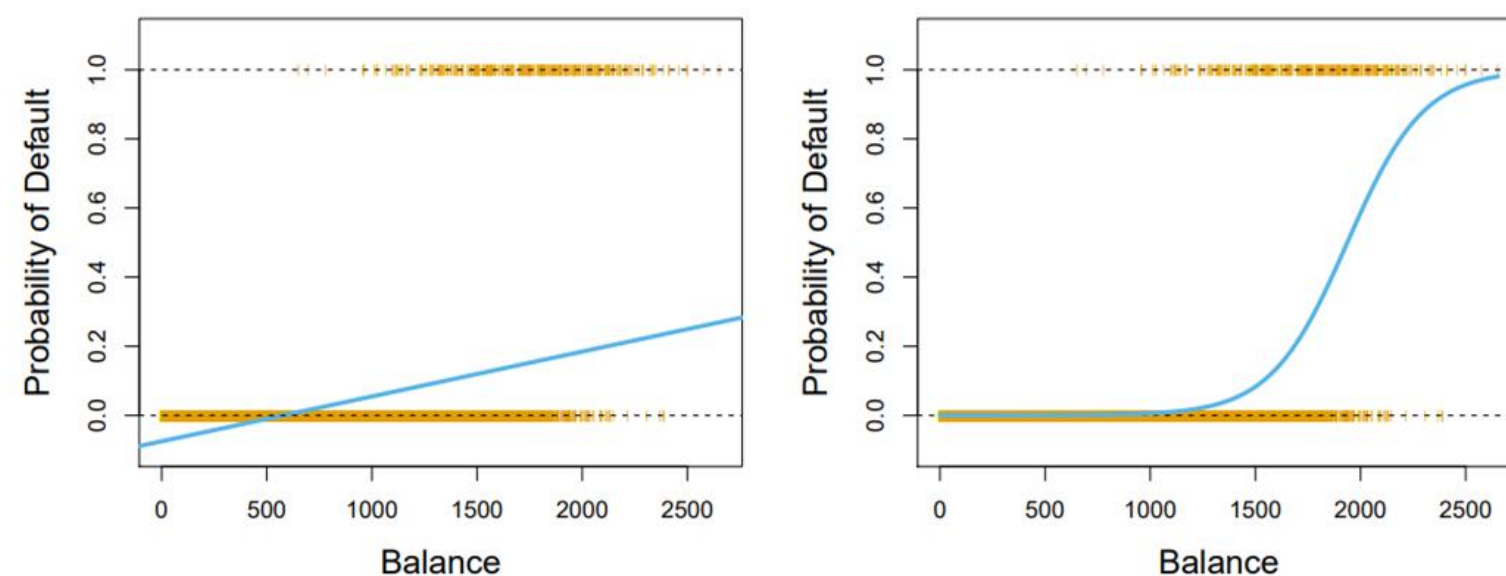


FIGURE 4.2. Classification using the `Default` data. Left: Estimated probability of `default` using linear regression. Some estimated probabilities are negative! The orange ticks indicate the 0/1 values coded for `default` (No or Yes). Right: Predicted probabilities of `default` using logistic regression. All probabilities lie between 0 and 1.

NAÏVE BAYES

$$P(y \mid x_1, \dots, x_n) = \frac{P(y)P(x_1, \dots, x_n \mid y)}{P(x_1, \dots, x_n)}$$

- Parte do pressuposto de que as variáveis explicativas/preditoras são independentes entre si (quando avaliadas par-a-par) – ou seja, ocorrem por eventos totalmente independentes;
- É probabilístico, ou seja, usa a probabilidade para fazer previsões;
- Assim, o modelo calcula a classe da observação dada a combinação de valores das variáveis preditoras;
- Quanto maior a correlação entre variáveis explicativas pior a performance do modelo.

PRINCIPAIS ALGORITMOS

● Classificação

● Regressão

REGRESSÃO
LOGÍSTICA



REGRESSÃO LINEAR,
L1 E L2



ÁRVORES



SVM



NAIVE BAYES



KNN



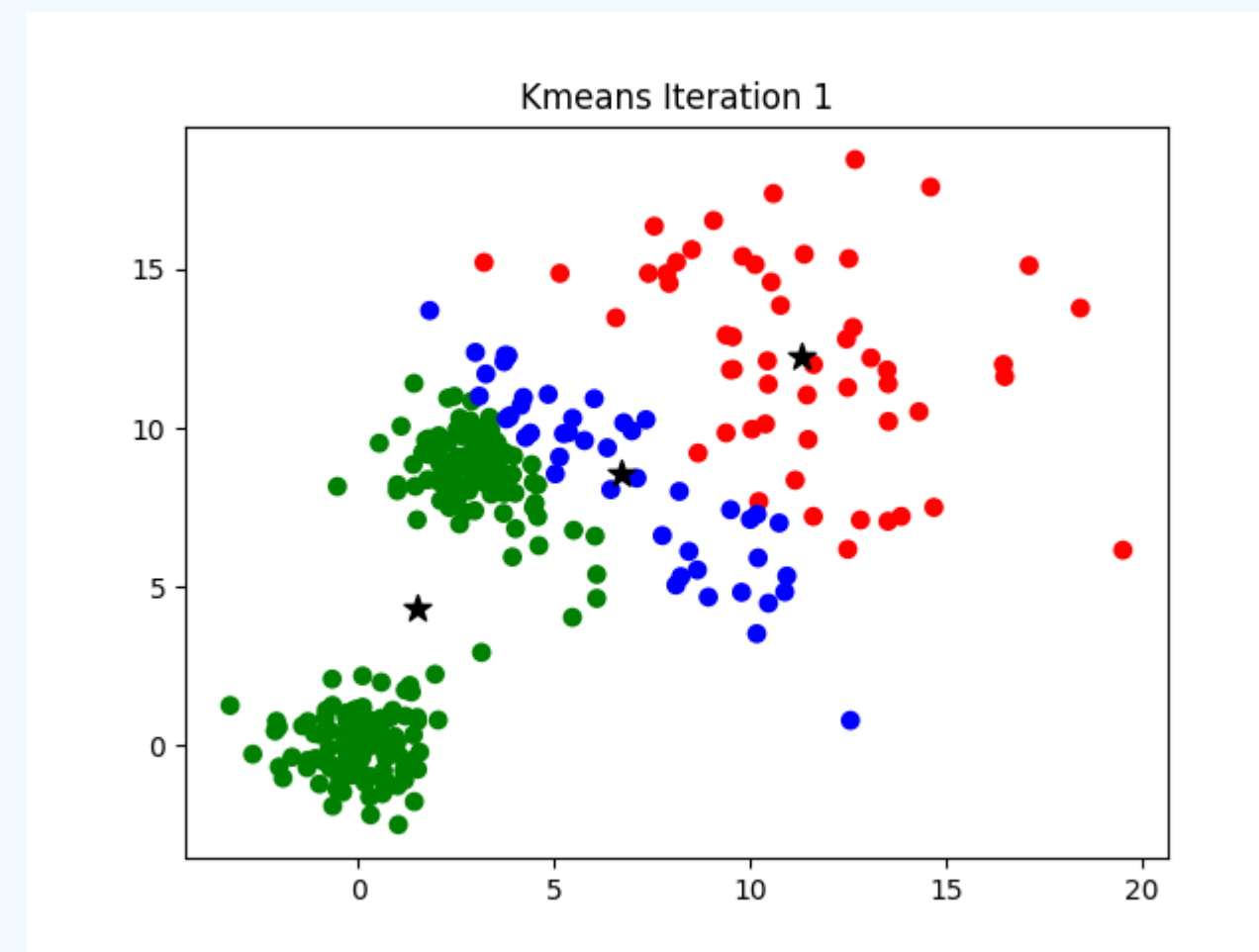
APRENDIZADO NÃO-SUPERVISIONADO

- É um tipo de aprendizado de máquina em que o algoritmo não é fornecido com dados rotulados.
- Podemos citar 3 tipos principais de aprendizado não-supervisionado:
 - Agrupamento (*clustering*)
 - Redução de Dimensionalidade
 - Aprendizado de Representações (autoencoders, GANs, etc.)
- As principais aplicações são:
 - Identificação de grupos/agrupamentos naturais dos dados
 - Detecção de anomalias
 - Visualização de dados

APRENDIZADO NÃO-SUPERVISIONADO

- Avaliar a saída do algoritmo é difícil: nem sempre o resultado condiz com a expectativa.
- Algoritmos não supervisionados são frequentemente usados em um ambiente exploratório, pois precisamos inspecionar manualmente os resultados para avaliar o desempenho do algoritmo.
- Pode ser aplicado como etapa de pré-processamento para algoritmos supervisionados. Isso pode melhorar a precisão dos algoritmos supervisionados ou reduzir o consumo de memória e tempo.

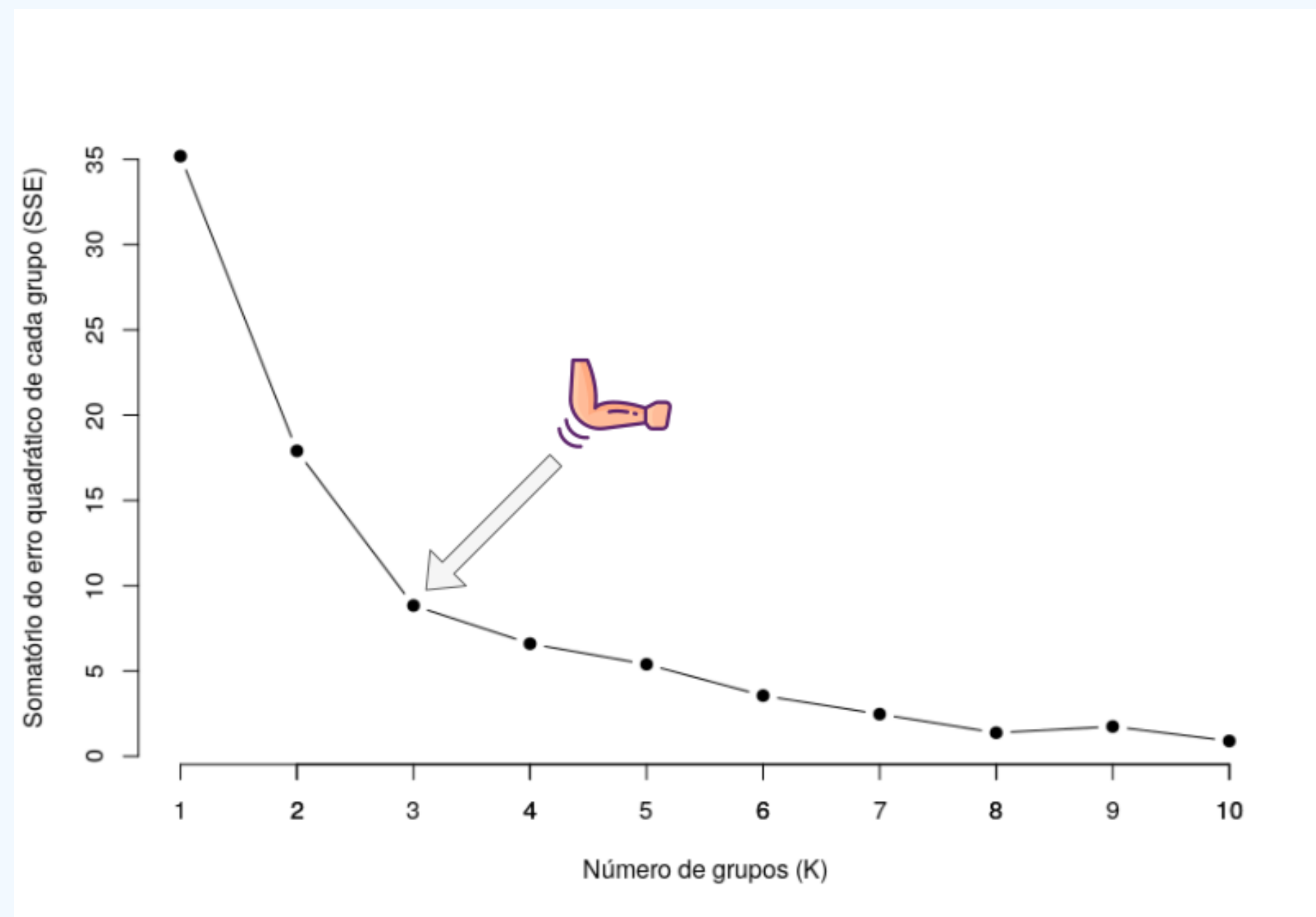
K-Means



Função Objetivo: O algoritmo resolve um problema de otimização para encontrar os centroides μ que minimizam a soma dos quadrados dentro do cluster (WCSS)

$$J = \sum_{j=1}^k \sum_{x \in C_j} \|x - \mu_j\|^2$$

Como encontrar K? Cotovelo



Procuramos o ponto de inflexão (o "cotovelo"). Antes desse ponto, adicionar clusters traz um ganho enorme de coesão; após esse ponto, o ganho de informação é marginal e você começa a sofrer com overfitting do agrupamento.

Como encontrar K? Silhueta

Este é um método mais sofisticado, pois avalia não apenas a coesão interna, mas também a **separação** entre os clusters.

Para cada ponto i , calculamos a silhueta $s(i)$:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Onde:

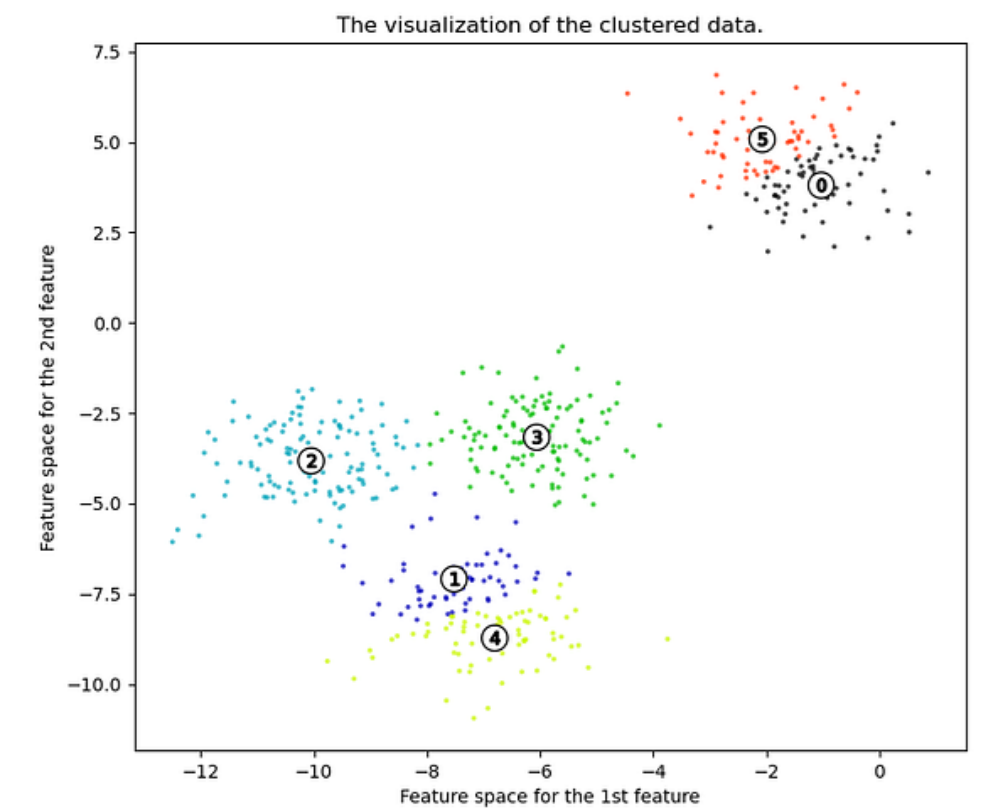
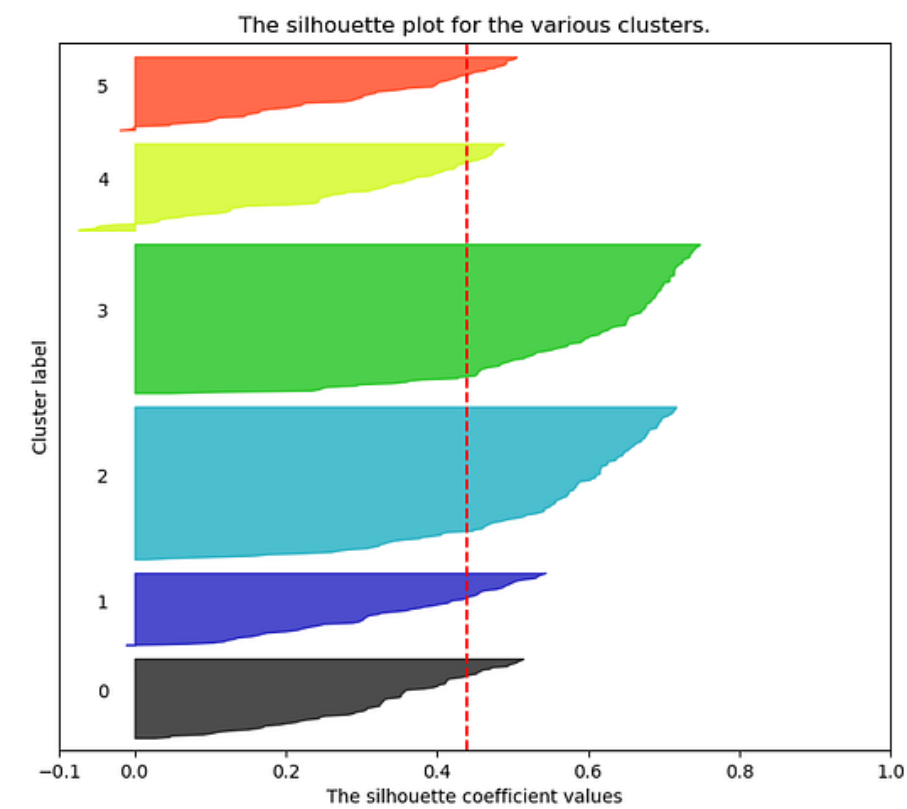
- $a(i)$: Distância média de i para todos os outros pontos do **mesmo** cluster (quão bem alocado ele está).
- $b(i)$: Distância média de i para os pontos do cluster vizinho **mais próximo** (quão bem separado ele está).

Interpretando os resultados:

- **Próximo de +1**: O ponto está muito bem agrupado e longe dos vizinhos.
- **Próximo de 0**: O ponto está na fronteira entre dois clusters.
- **Próximo de -1**: O ponto provavelmente foi atribuído ao cluster errado.

Como encontrar K? Silhueta

Silhouette analysis for KMeans clustering on sample data with $n_clusters = 6$

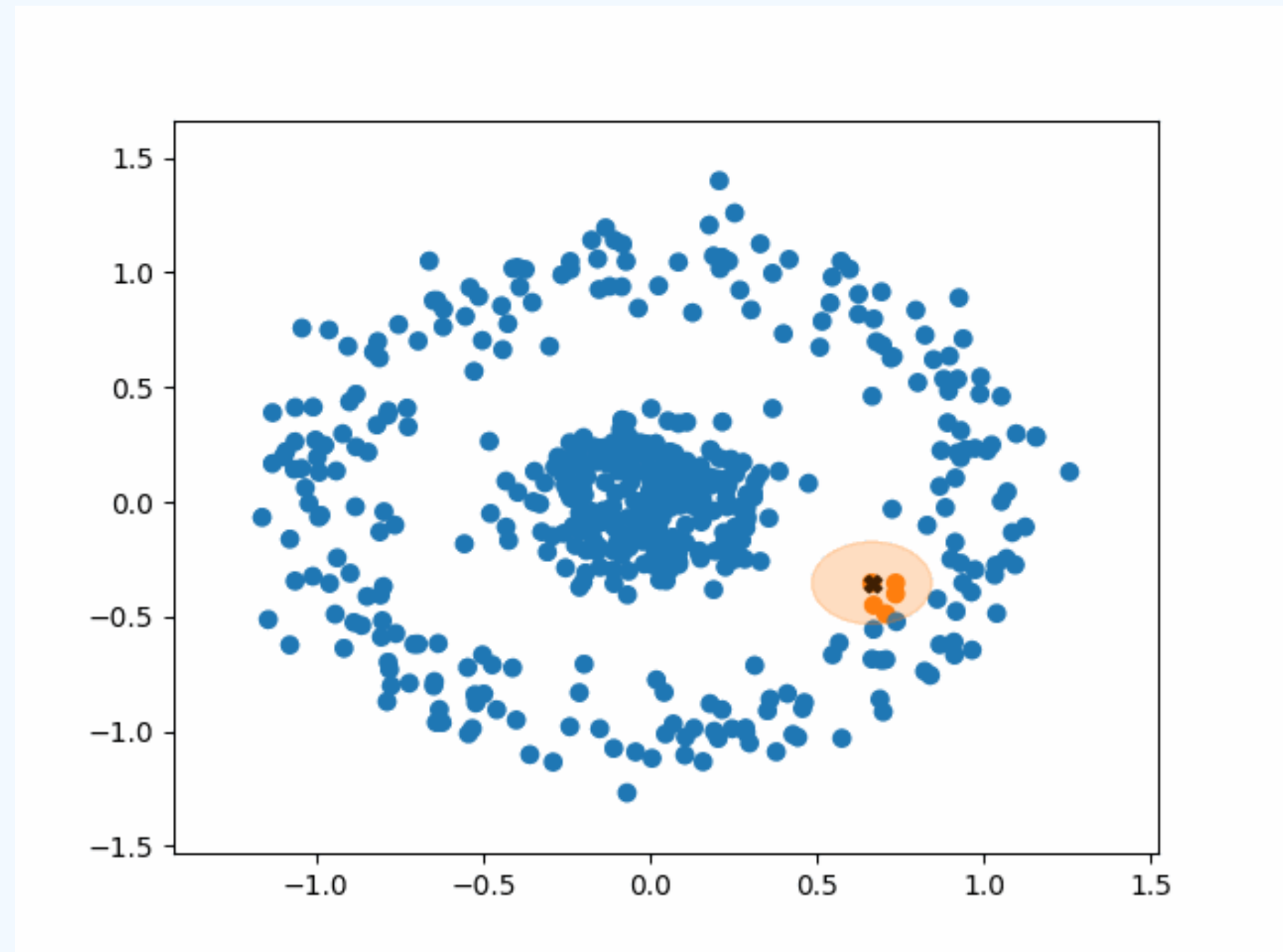


DBSCAN

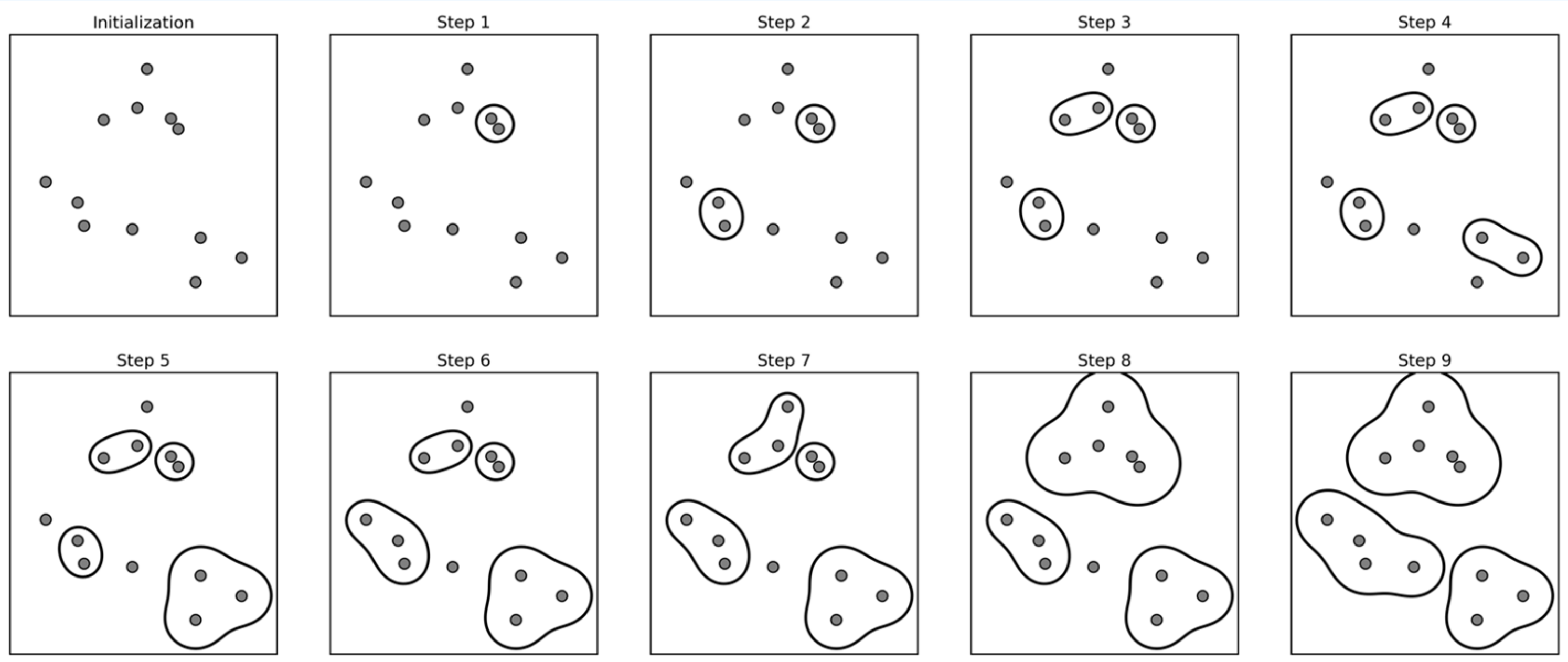
O DBSCAN (Density-Based Spatial Clustering of Applications with Noise) é excelente para encontrar formas arbitrárias e identificar outliers.

- Parâmetros Fundamentais:
 - Eps: O raio da vizinhança.
 - MinPts: O número mínimo de pontos para formar uma "região densa".
- Lógica de Classificação:
 - Core Points: Têm MinPts dentro de Eps.
 - Border Points: Estão no raio de um Core Point, mas têm poucos vizinhos.
 - Noise (Ruído): Não são nem core nem border.
- Vantagem: Não precisa definir o número de clusters a priori e lida bem com ruído.

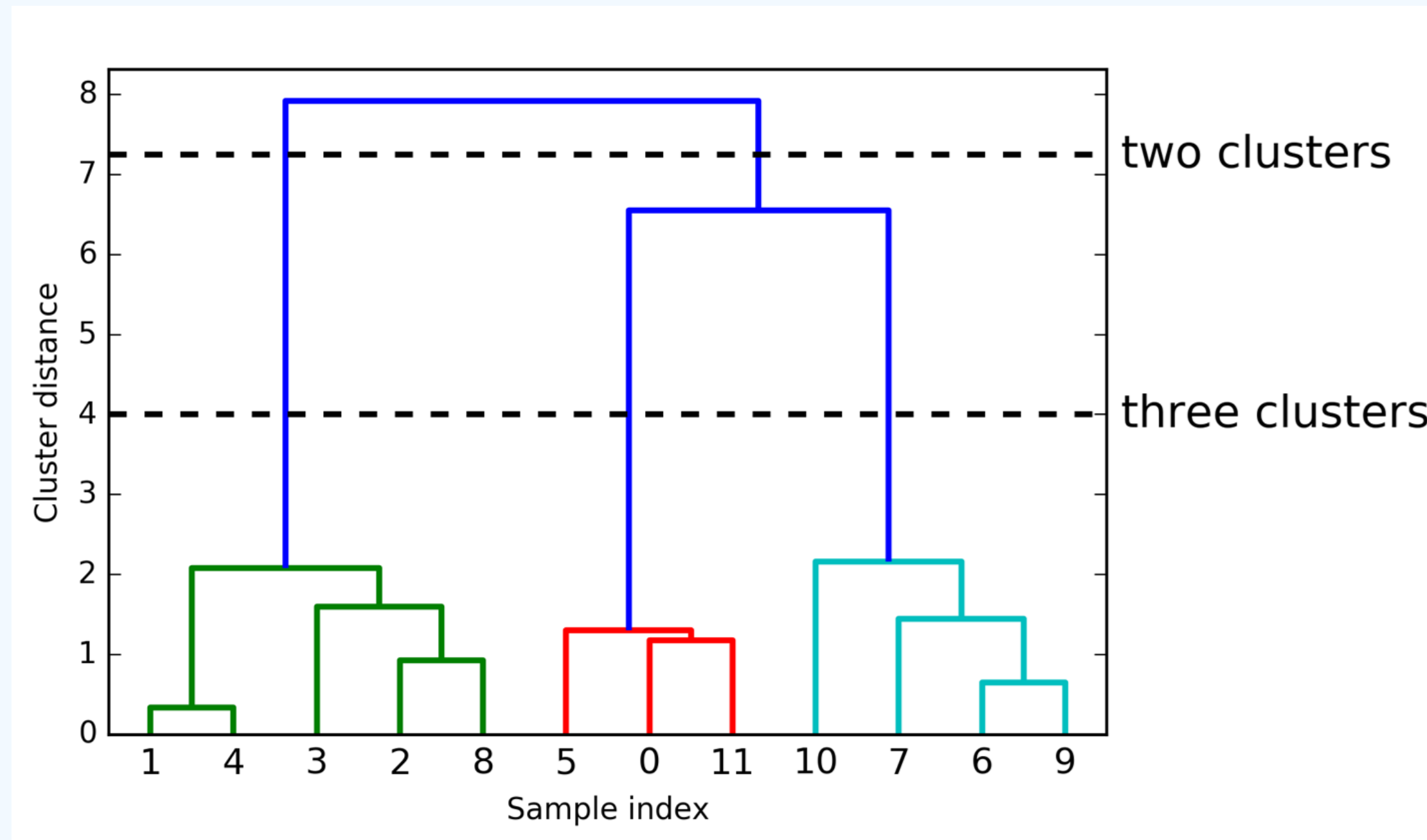
DBSCAN



CLUSTERIZAÇÃO AGGLOMERATIVE



CLUSTERIZAÇÃO HIERÁRQUICO



CLUSTERIZAÇÃO HIERÁRQUICO

